Emotional Models: Supervised vs. Zero-Shot Classification

Miguel Castela[1] and Miguel Martins[2]

Univsersity of Coimbra, Portugal
uc2022212972@student.uc.pt
uc2022213951@student.uc.pt

1 Introduction

This project presents the implementation and evaluation of two distinct approaches to emotion classification in text using Transformer-based models applied to the *Emotion* dataset. The Emotion dataset, available on the HuggingFace Hub under the identifier `dair-ai/emotion`, consists of short English text samples primarily collected from social media platforms.

The dataset is pre-split into training, validation, and test sets. The main objective of this project is to compare the performance of two different methodologies for emotion classification. The main objective of this project is to compare the performance of two different methodologies for emotion classification. The first approach follows a supervised learning paradigm, where pretrained Transformer models are used as frozen feature extractors to generate sentence embeddings via models from the `sentence-transformers` library. The second approach adopts a zero-shot learning strategy based on Natural Language Inference (NLI) models, where emotion labels are expressed as natural language hypotheses and classification is performed without task-specific fine-tuning. By evaluating both supervised and zero-shot methods on the same dataset, this project aims to analyze their respective strengths and limitations, as well as characteristic error patterns.

Additionally, the impact of different pretrained models from the HuggingFace Hub is explored, assessing how variations in model size, architecture, and training objectives affect classification performance.

2 Description of the problem

The `dair-ai/emotion` dataset consists of text samples, primarily collected from Twitter, labeled with one of six emotions: *joy, sadness, anger, fear, love* and *surprise*.The training set containing 16,000 samples, the validation set 2,000 samples, and the test set 2,000 samples. The samples are distributed according to the following table:

Table 1. Class distribution of the `dair-ai/emotion` dataset.

Emotion	Train	Validation	Test	Total (%)
Joy	5,362	704	695	33.8%
Sadness	4,666	550	581	29.0%
Anger	2,159	275	275	13.5%
Fear	1,937	212	224	11.9%
Love	1,304	178	159	8.2%
Surprise	572	81	66	3.6%
Total	**16,000**	**2,000**	**2,000**	**100.0%**

By looking at table 1, it's clear that the dataset is imbalanced, with the *joy* and *sadness* classes being significantly more represented, together accounting for over 60% of the data. In contrast, the *surprise* class is the least represented by a considerable margin, 3.6%. This imbalance could lead to biased model performance, favoring the majority classes, and makes it clear that "Accuracy" might not be the best metric to evaluate the models.

Another issue present is the overfitting of words or terms to certain emotions. For example, the word "happy" is strongly associated with the *joy* class, and its presence in a text sample could lead the model to predict *joy* regardless of the overall context. One example of this is [por exemplo...]. Such biases can hinder the model's ability to generalize to unseen data, especially when the emotional context is more nuanced or when words have different connotations in different contexts.

3 Approach and Architecture

3.1 Supervised Classification with Frozen Transformer Embeddings

This approach follows a supervised learning paradigm in which pretrained Transformer models are used as fixed feature extractors, and a lightweight neural classifier is trained on top of the resulting sentence embeddings. The model leverages their pretrained semantic representations to reduce computational cost and mitigate overfitting, while still capturing rich contextual information from the input text.

The Emotion dataset is loaded using the HuggingFace `datasets` library and split into the training, validation, and test sets. To convert raw text into numerical representations suitable for classification, a pretrained sentence-level Transformer from the `sentence-transformers` library is employed.

3.2 Embedding Generation

Specifically, the models `all-MiniLM-L6-v2` and `distilroberta-base-paraphrase-v1`

are used to encode each sentence into a dense, fixed-length embedding vector. The embedding process is performed in batches of 64 samples to improve efficiency, and the resulting representations are stored as NumPy arrays for the training, validation, and test sets. The Transformer remains frozen during this stage, meaning that no gradient updates are applied to its parameters.

These embeddings serve as high-level semantic features that capture syntactic and contextual information from the input text. The corresponding emotion labels are retained as integer class indices, resulting in pairs of the form *(embedding, label)*, which are later wrapped into PyTorch `DataLoader` objects for mini-batch training and evaluation.

Classifier Architecture
On top of the generated embeddings, we trained a non-linear classifier (Multi-Layer Perceptron).The classification head is implemented as a shallow Artificial Neural Network (ANN), equivalent to a small Multi-Layer Perceptron. Its architecture is intentionally lightweight, as the main representational power is delegated to the pretrained Transformer embeddings. The architecture, defined in our `ANNClassifier` module, consists of:

- **Input Layer:** Matches the dimension of the embeddings (384 dimensions for MiniLM).
- **Hidden Layer:** A fully connected layer with 256 neurons, followed by a Rectified Linear Unit (ReLU) activation function, introducing non-linearity into the model.
- **Regularization:** A Dropout layer with a probability of $p = 0.1$ to prevent overfitting.
- **Output Layer:** A fully connected layer with six neurons, one for each emotion class, producing unnormalized logits.

Class predictions are obtained by applying the `argmax` operation over the output logits, selecting the emotion class with the highest predicted score.

3.3 Training Procedure

The classifier is trained using the Adam optimizer with a learning rate of 10^{-3} and the Cross-Entropy loss function, which is standard for multi-class classification tasks. Training is performed using mini-batch gradient descent, where the embedding–label pairs are processed in batches rather than as a whole. This improves computational efficiency and introduces beneficial stochasticity into the optimization process.

To prevent overfitting and ensure robust model selection, early stopping is employed based on the macro-averaged F1 score computed on the validation set. Training is halted if no improvement in validation F1 is observed for a fixed number of consecutive epochs, and the best-performing model state is restored before final evaluation.

3.4 Evaluation

After training, the selected model is evaluated on the held-out test set. Performance is measured using accuracy, macro-averaged F1 score, and confusion matrices. To account for randomness introduced by weight initialization and data shuffling, experiments are repeated across multiple random seeds, and results are reported as mean values with corresponding standard deviations. This evaluation protocol enables a fair and reliable assessment of the supervised approach based on frozen Transformer embeddings.

3.5 Zero-Shot Classification via Natural Language Inference (NLI)

The zero-shot classification approach leverages pre-trained Natural Language Inference (NLI) models to classify text without specific task-related training data. NLI models are trained to determine the relationship (entailment, contradiction, or neutral) between a premise and a hypothesis.

- **Hypothesis Template:** The zero-shot classification task requires formulating the candidate label into a structured hypothesis sentence. The chosen template is `"This text expresses ."` (`NLI.py`).

Classification Process

1. **Pipeline Initialization:** A `transformers` pipeline for `"zero-shot-classification"` is initialized with the chosen NLI model and automatically placed on the available device.
2. **Batch Inference:** The test texts are processed in batches of 32 for computational efficiency.
3. **Prediction:** For each text, the model compares the text (premise) against a hypothesis for every candidate label.
4. **Label Selection:** The label whose hypothesis yields the highest entailment score is selected as the predicted class for the input text.

Evaluation Metrics

The performance of the zero-shot classifier is evaluated on the test set using standard classification metrics:

Unlike the supervised pipeline, this method was applied directly to the test set without any parameter updates.

3.6 Evaluation

After obtaining predictions for the test set, the model's performance is assessed using accuracy, macro-averaged F1 score, and confusion matrices. Similar to the supervised approach, experiments are repeated across multiple random seeds to ensure robustness, and results are reported as mean values with standard deviations. This evaluation framework allows for a comprehensive comparison between the zero-shot NLI-based method and the supervised classification approach using frozen Transformer embeddings.

4 Experimental Setup

In this section, we detail the specific models employed in both the supervised and zero-shot classification approaches, along with their configurations and relevant hyperparameters.

4.1 Supervised Models

The ANN utilized in supervised training was configured with the hyperparameters listed in Table 2. Two distinct pretrained Transformer models from the `sentence-transformers` library, mentioned in section 3.2 were employed to generate sentence embeddings: `distilroberta-base-paraphrase-v1` and `all-MiniLM-L6-v2`. Their configurations, including embedding dimensions and batch sizes, are summarized in Table 3. Experiments were conducted using five different random seeds to account for variability in training outcomes.

Table 2. Hyperparameters for supervised training

Hyperparameter	**Value**
Batch Size	64
Learning Rate	1×10^{-3}
Epochs	25 with Early Stopping
Loss Function	Cross-Entropy Loss
Optimizer	Adam
Seeds	[2025, 2029]

Table 3. Supervised Models and their configurations.

Model	**Embedding Dimension**	**Batch Size**
all-MiniLM-L6-v2	384	64
distilroberta-base-paraphrase-v1	768	64
seeds	[2025,2029]	

5 Results and Discussion

5.1 Supervised Models

The following table (Table 4) summarizes the performance across on the different

Model	Head Type	Mean Accuracy	Mean Macro F1	Avg. Epochs
all-MiniLM-L6-v2	ANN	0.7315	0.6556	20.0
all-MiniLM-L6-v2	LogReg	0.6870	0.5928	N/A
paraphrase-mpnet-base-v2	ANN	0.7089	0.6249	12.4
paraphrase-mpnet-base-v2	LogReg	0.6950	0.6083	N/A
all-mpnet-base-v2	ANN	0.7014	0.6177	13.0
all-mpnet-base-v2	LogReg	0.6760	0.5629	N/A

Table 4. Performance Comparison of Embedding Models and Classifiers

Table 5. Zero-Shot Classification Results

Model Name	ACC	Macro F1
MoritzLaurer/xtremedistil-l6-h256-zeroshot-v1.1-all-33	0.6120	0.5148
joeddav/xlm-roberta-large-xnli	0.3505	0.3222
MoritzLaurer/bge-m3-zeroshot-v2.0	0.6960	0.6232
MoritzLaurer/deberta-v3-xsmall-zeroshot-v1.1-all-33	0.7045	0.6306
MoritzLaurer/ModernBERT-large-zeroshot-v2.0	0.7090	0.6304
facebook/bart-large-mnli	0.4660	0.4333

NLI Models

5.2 Supervised vs Zero-Shot Classification

5.3 NLI Models

Theorem 1. *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

Proof. Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [1], an LNCS chapter [2], a book [3], proceedings without editors [4], and a homepage [5]. Multiple citations are grouped [1–3], [1, 3–5].

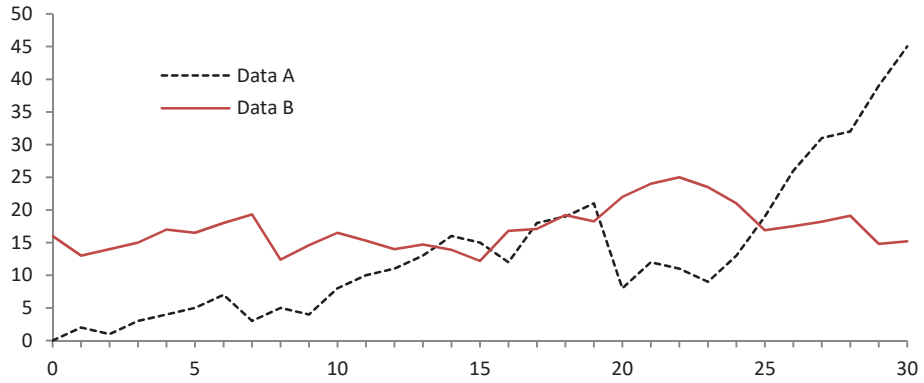

Fig. 1. A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

Acknowledgments. A bold run-in heading in small font size at the end of the paper is used for general acknowledgments, for example: This study was funded by X (grant number Y).

Disclosure of Interests. It is now necessary to declare any competing interests or to specifically state that the authors have no competing interests. Please place the statement with a bold run-in heading in small font size beneath the (optional) acknowledgments[1], for example: The authors have no competing interests to declare that are relevant to the content of this article. Or: Author A has received research grants from Company W. Author B has received a speaker honorarium from Company X and owns stock in Company Y. Author C is a member of committee Z.

References

1. Author, F.: Article title. Journal **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) CONFERENCE 2016, LNCS, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). https://doi.org/10.10007/1234567890
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, http://www.springer.com/lncs, last accessed 2023/10/25

[1] If EquinOCS, our proceedings submission system, is used, then the disclaimer can be provided directly in the system.